

Conditional sync in action

- Let's consider the below class, and define
 - **final state** as any state where no thread can make progress
 - notation for thread that executes $A_1, \dots, A_n$: $T = \{A_1, \dots, A_n\}$
 - a SecretHolder instance I

```
public class SecretHolder {
    private int secret = 0;
    private final int MAX_VALUE = 5;

    private synchronized void checkSecret(){
        if (secret < 0 || secret > MAX_VALUE) {
            throw new AssertionError("Error!");
        }
    }
    ...
}
```

Conditional sync in action [Qa]

- Suppose $A = \{l.\text{downOne}()\}$ and $B = \{l.\text{upOne}()\}$
- What values can $l.\text{secret}$ have in final states?

```
public class SecretHolder {  
    ...  
    public synchronized void upOne() {  
        if (secret==MAX_VALUE) {wait();}  
        secret++; checkSecret(); notify();  
    }  
  
    public synchronized void downOne() {  
        if (secret==0) {wait();}  
        secret--; checkSecret(); notify();  
    }  
    ...  
}
```

Conditional sync in action [Qa]

- Suppose $A = \{l.downOne()\}$ and $B = \{l.upOne()\}$
- What values can $l.secret$ have in final states?

```
public class SecretHolder {
    ...
    public synchronized void upOne() {
        if (secret==MAX_VALUE) {wait();}
        secret++; checkSecret(); notify();
    }

    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }
    ...
}
```

Only two possible interleavings

Interleaving #1:
 upOne(): secret=1
 downOne(): secret=0

Interleaving #2:
 downOne(): wait
 upOne(): secret=1
 downOne(): secret=0

Conditional sync in action [Qb]

- Suppose $A = \{l.\text{downOne}()\}$ and $B = \{l.\text{upOne}()\}$
- Can an `AssertionError` be thrown by `checkSecret()`?

```
public class SecretHolder {  
    ...  
    public synchronized void upOne() {  
        if (secret==MAX_VALUE) {wait();}  
        secret++; checkSecret(); notify();  
    }  
  
    public synchronized void downOne() {  
        if (secret==0) {wait();}  
        secret--; checkSecret(); notify();  
    }  
    ...  
}
```

Conditional sync in action [Qb]

- Suppose $A = \{l.\text{downOne}()\}$ and $B = \{l.\text{upOne}()\}$
- Can an `AssertionError` be thrown by `checkSecret()`?

```
public class SecretHolder {
    ...
    public synchronized void upOne() {
        if (secret==MAX_VALUE) {wait();}
        secret++; checkSecret(); notify();
    }

    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }
    ...
}
```

Nope :-)

See previous slide
for the two possible
interleavings.

Conditional sync in action [Qc]

- Suppose $A = \{l.\text{downOne}()\}$, $B = \{l.\text{downOne}()\}$, $C = \{l.\text{downOne}()\}$, $D = \{l.\text{upTwo}()\}$
- Can an `AssertionError` be thrown by `checkSecret()`?

```
public class SecretHolder {  
    ...  
    public synchronized void downOne() {  
        if (secret==0) {wait();}  
        secret--; checkSecret(); notify();  
    }  
  
    public synchronized void upTwo() {  
        while (secret>=MAX_VALUE-1) {wait();}  
        secret+=2; checkSecret(); notifyAll();  
    }  
    ...  
}
```

Conditional sync in action [Qc]

- Suppose $A = \{l.downOne()\}$, $B = \{l.downOne()\}$, $C = \{l.downOne()\}$, $D = \{l.upTwo()\}$
- Can an `AssertionError` be thrown by `checkSecret()`?

```
public class SecretHolder {
    ...
    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }

    public synchronized void upTwo() {
        while (secret>=MAX_VALUE-1) {wait();}
        secret+=2; checkSecret(); notifyAll();
    }
    ...
}
```

Interleaving:

- A, B, C all block on `wait()`
- D sets `secret=2`, and `notifyAll()`
- A, B, C wake up (unknown order)
- last among A, B, and C sets `secret = -1`

Conditional sync in action [Qc]

- Suppose $A = \{l.\text{downOne}()\}$, $B = \{l.\text{downOne}()\}$, $C = \{l.\text{downOne}()\}$, $D = \{l.\text{upTwo}()\}$
- Can an `AssertionError` be thrown by `checkSecret()`?

```
public class SecretHolder {
    ...
    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }

    public synchronized void upTwo() {
        while (secret>=MAX_VALUE-1) {wait();}
        secret+=2; checkSecret(); notifyAll();
    }
    ...
}
```

So, yes! :-)

Problem:
`downOne()` does
 not check condition
 predicate after wait
 (*do NOT do that!*)

Conditional sync in action [Qd]

- Let $A=\{I.\text{downOne}()\}$, $B=\{I.\text{upOne}()\}$, $C=\{I.\text{upTwo}()\}$ and $I.\text{secret}=5$. Is $I.\text{secret}=4$ in any final state?

```
public class SecretHolder {
    public synchronized void upOne() {
        if (secret==MAX_VALUE) {wait();}
        secret++; checkSecret(); notify();
    }
    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }
    public synchronized void upTwo() {
        while (secret>=MAX_VALUE-1) {wait();}
        secret+=2; checkSecret(); notifyAll();
    }
}
...
```

Conditional sync in action [Qd]

- Let $A=\{I.downOne()\}$, $B=\{I.upOne()\}$, $C=\{I.upTwo()\}$ and $I.secret=5$. Is $I.secret=4$ in any final state?

```
public class SecretHolder {
    public synchronized void upOne() {
        if (secret==MAX_VALUE) {wait();}
        secret++; checkSecret(); notify();
    }
    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }
    public synchronized void upTwo() {
        while (secret>=MAX_VALUE-1) {wait();}
        secret+=2; checkSecret(); notifyAll();
    }
}
...
```

Interleaving:

- B and C both block on wait()
- A sets secret=4, and notify()
- C wakes up, but gets back to wait()
- B never woke up

Conditional sync in action [Qd]

- Let $A=\{l.downOne()\}$, $B=\{l.upOne()\}$, $C=\{l.upTwo()\}$ and $l.secret=5$. Is $l.secret=4$ in any final state?

```
public class SecretHolder {
    public synchronized void upOne() {
        if (secret==MAX_VALUE) {wait();}
        secret++; checkSecret(); notify();
    }
    public synchronized void downOne() {
        if (secret==0) {wait();}
        secret--; checkSecret(); notify();
    }
    public synchronized void upTwo() {
        while (secret>=MAX_VALUE-1) {wait();}
        secret+=2; checkSecret(); notifyAll();
    }
}
...
```

So, yes...

Problem: downOne()
wakes up only one
thread

Note: can be seen as
a missed signal

Conditional sync in action [Qe]

- Suppose any number of {l.downTwo()} and any number of {l.upTwo()}
- Can an AssertionError be thrown at runtime?

```
public class SecretHolder {  
    ...  
    public synchronized void upTwo() {  
        while (secret >= MAX_VALUE - 1) {wait();}  
        secret += 2; checkSecret(); notifyAll();  
    }  
  
    public synchronized void downTwo() {  
        while (secret <= 1) {wait();}  
        secret -= 2; checkSecret(); notifyAll();  
    }  
    ...  
}
```

Conditional sync in action [Qe]

- Suppose any number of {l.downTwo()} and any number of {l.upTwo()}
- Can an AssertionError be thrown at runtime?

```
public class SecretHolder {
    ...
    public synchronized void upTwo() {
        while (secret >= MAX_VALUE - 1) {wait();}
        secret += 2; checkSecret(); notifyAll();
    }

    public synchronized void downTwo() {
        while (secret <= 1) {wait();}
        secret -= 2; checkSecret(); notifyAll();
    }
    ...
}
```

Nope :-)

Reason:

- Conditions always checked after wait()
- All threads notified after secret's value changes